INDIAN INSTITUE OF TECHNOLOGY MADRAS

HOUSEHOLD SERVICES

Modern Application Development 2 Project

Pranathi Ayyadevara

Introduction:

Household services application is a multi-user interactive platform for customers to book services and professionals to fulfill the requests. The application is built using latest technologies with responsive UI so customers can use the application on any device (phone, tab or computer).

Methodology:

Tools and technologies used:

Flask for main application container

SQLite for database storage

Flask-SQLAlchemy for database connection

Flask-Security for login (authentication) and roles (authorization)

Flask-Restful for API

Redis for caching

Celery for scheduling background task (email alerts) and asynchronous tasks (csv data downloads)

Mailhog for email alerts

Jinja2 templates for HTML email reports

Bootstrap, Vue and CSS for front end application

Application Functionality:

Following are few details of the application:

The application has three types of roles, Administrator (only one user), Professionals and Customers.

Professional and customers can register to the application via registration page.

Professionals have to register for a specific Service category with their years of experience. Upon review, Administrator can approve Professional and activate them to use the system. Customers are activated by default.

Upon login, Administrator, Professionals and Customers land on their respective Dashboards. The menu system is dynamically updated based on their login credentials.

Administrator can deactivate Professional or a Customer based on adherence to company policies in the User management module.

Administrator can perform various service-related tasks in Service management module like, Add Service category and Add/Update/Delete Services under a Service category. For example, if house cleaning is a Service category, room cleaning, bathroom cleaning, attic cleaning are all services under that category.

Customers can browser all services, filter them by service category and if interested, can book a Service for a specific date and provide instructions.

Each booking goes through a life cycle. Customer can update the service booking before it is confirmed by a Professional. Once confirmed by a Professional, the booking can't be edited any further. They will also receive an email alert of their booking.

All Professionals belong to a specific Service category can see all Customer requests for that Service category. If they are interested, they can accept the service request. A booking confirmation alert is sent to both Customer and Professionals.

Customer can close the booking once service is completed. Both Professional and Customer will get an email alert.

Customer can pay for the service using Payment page. Currently it is a dummy page as a place holder.

Customer can provide rating to the Professional for the closed booking. He/she can rate between 1 and 5, 1 being awful and 5 being outstanding. An email alert will be sent to both Professional and Customer upon the rating. Once a booking is rated, the status will be complete and no further action can be performed on the booking.

Customer can see summary graphs of their bookings (by status), ratings they provided to the Professionals and the Services they ordered.

Professionals can see summary graphs of their current bookings (by status), ratings they received and the Services they performed.

Administrator can see summary and all bookings, ratings of all Professionals, the count of Services by their type.

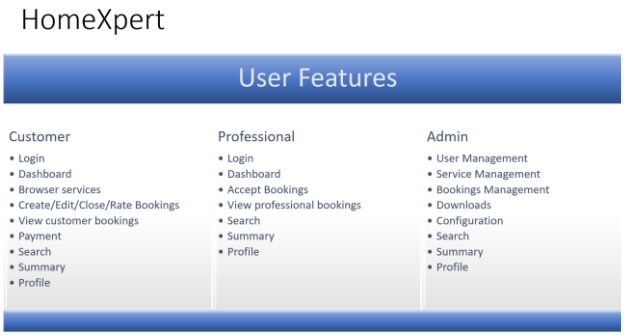
Search functionality for Service Category and Services is provided to all users of the system. In addition, Administrator can search for all users and their statuses.

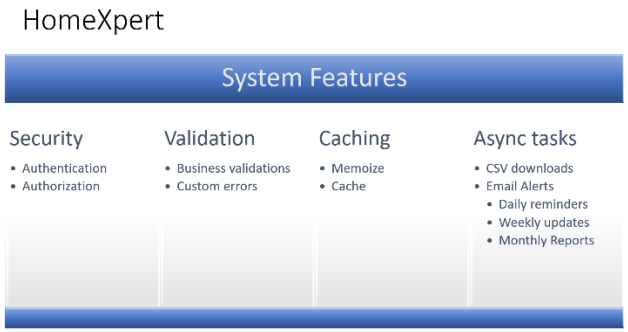
All users can view their profile. They can update certain fields like password, area code, address, and contact numbers.

Administrator can view all bookings in the Booking Management module

Administrator can download all data (users, service categories, services, bookings, reviews) in CSV format. The download tasks are performed asynchronously using celery

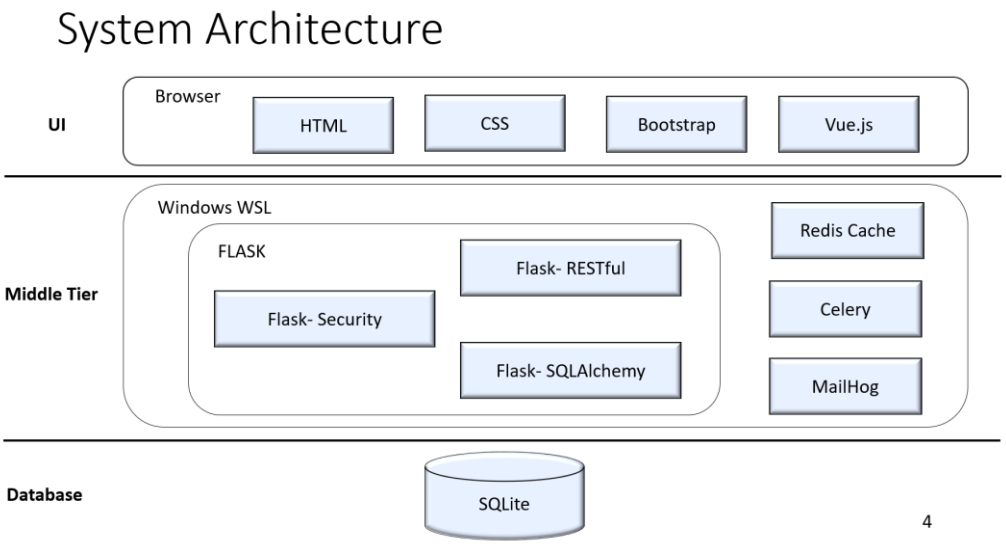
Administrator can change the system configuration like the time for daily alerts, day of week for weekly updates and day of the month for monthly reports. Administrator can also configure Memoize and Cache timeouts in the configuration module. (Note: Frontend screen is completed for concept as proof of concept, backend persistence is not done at this time)





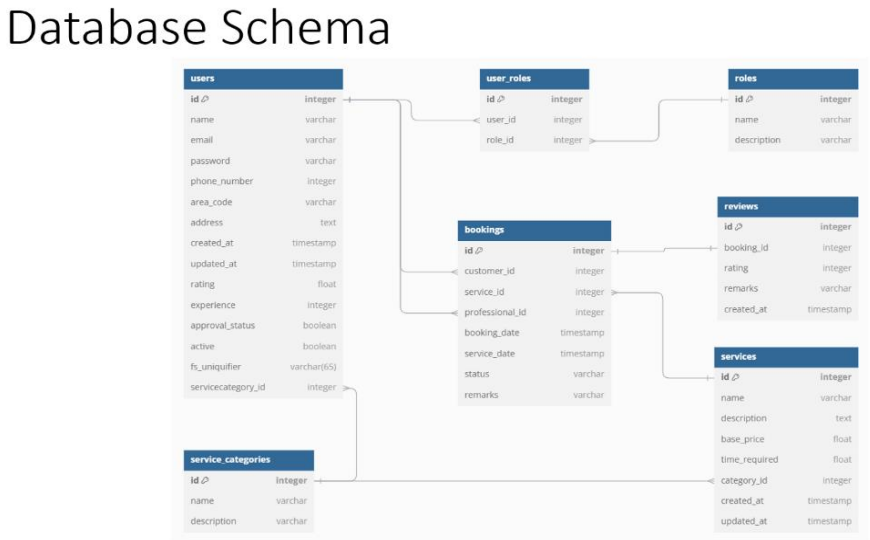
Architecture:

The client accesses the app from a web browser like Edge, Firefox, Chrome, Brave, etc. The application is a **single page** application. Following is the three-tier system architecture.



Database Schema:

The database has 7 tables, Users, Roles, User-Role mapping, Service categories, Services, Bookings and Reviews. Relationships between tables is described in the following diagram:



Security:

User passwords are hashed and persisted in the database so no one can view them directly.

Authentication token is generated on the server and each client request should contain the token (except for home page, login page and registration page). The token timeout could be configured by the system administrator during deployment time.

Web service API's are protected for token authentication and Role based authorization

API Routes:

Following are the backend API routes:

API Routes

```
LoginAPI, "/login"
RegisterAPI, "/register"

UserListAPI, "/users"
UserAPI, "/users/<int:user_id>"
UserSearchAPI, "/users/search/<search_query>"

ServiceCategoryListAPI, "/servicecategories"
ServiceCategoryAPI,
"/servicecategories/<int:service_category_id>"
ServiceCategorySearchAPI,
"/servicecategories/search/<search_query>"

ServiceListAPI, "/services"
ServiceAPI, "/services/<int:service_id>"
ServiceSearchAPI,
"/services/search/<search_query>"

BookingListAPI, "/bookings"
BookingAPI, "/bookings/<int:booking_id>"

ReviewListAPI, "/reviews"
ReviewAPI, "/reviews/<int:review_id>"

ServiceGraphsAPI, "/graph/servicebookings"
BookingGraphAPI, "/graph/bookingstatus"
ReviewGraphAPI, "/graph/reviews"

CreateFileAPI,
"/download/create/<string:artifact_type>"
CheckFileStatusAPI,
"/download/checkstatus/<string:task_id>"
DownloadFileAPI, "/download/file/<string:task_id>"
```

Frontend Routes:

Following are the Frontend Vue routes:

Frontend Routes

```
{path : "/", component : HomePage},
{path : "/login", component : LoginPage},
{path : "/register/:registerFor", props : true, component : RegisterPage},
{path : "/adminDashboard", component : AdminDashBoardPage, meta : {requiresLogin : true, role :
"admin"}},
{path : "/professionalDashboard", component : ProfessionalDashboardPage, meta : {requiresLogin : true,
role : "professional"}},
{path : "/customerDashboard", component : CustomerDashboardPage, meta : {requiresLogin : true, role :
"customer"}},
{path : "/search", component : SearchPage, meta : {requiresLogin : true}},
{path : "/summary", component : SummaryPage, meta : {requiresLogin : true}},
{path : "/profile", component : ProfilePage, meta : {requiresLogin : true}},
{path : '/reviews/:id', component : DisplayReviewPage, props : true, meta : {requiresLogin : true}},
{path : "/users", component : UserList, meta : {requiresLogin : true}},
{path : "/services", component : Services, meta : {requiresLogin : true}},
{path : "/bookings", component : BookingList, meta : {requiresLogin : true}},
{path : "/downloads", component : Downloads, meta : {requiresLogin : true}},
{path : "/configuration", component :Configuration, meta : {requiresLogin : true}},
{path : "/payment", component : PaymentPage, meta : {requiresLogin : true, role : "customer"}}
```

Application Components:

Following are backend components python files:

Backend Components

General	Celery	API	Utils	Instance
app.py	celery_factory.py	booking.py	constants.py	database.sqlite3
config.py	celery_schedule.py	download.py	seed.py	booking.csv
init_db.py	mail_service.py	graph.py	utilities.py	review.csv
models.py	tasks.py	review.py		service.csv
routes.py		resources.py		service_category.csv
requirements.txt		service.py		user.csv
		servicecategory.py		
		user.py		

Following are frontend components, pages, images, templates (for emails) and utility functions:

Frontend Components

Components	Pages	Images	Templates	Utilities
BookingDetails.js	AdminDashBoardPage.js	component-gears.png	base.html	router.js
BookingList.js	CustomerDashboardPage.js	components-card.png	booking.html	store.js
ComponentModal.js	DisplayReviewPage.js	demand_house.png	daily_reminder.html	
Configuration.js	HomePage.js	favicon.ico	index.html	
Downloads.js	LoginPage.js	hss_background.jpg	monthly_report.html	
GenericCard.js	ProfessionalDashboardPage.js	hss_logo.jpg		
GenericDialogue.js	ProfilePage.js	ionic-native-card.png		
Navbar.js	RegisterPage.js	retailer_house.png		
PopupModal.js	SearchPage.js	slide-1-house.jpg		
ReviewDetails.js	SummaryPage.js	slide-2-house.jpg		
ServiceCategoryDetails.js		slide-3-house.jpg		
ServiceCategoryList.js		theming-card.png		
ServiceDetails.js				
ServiceList.js				
Services.js				
UserList.js				

Project Demo Video Link:

https://drive.google.com/drive/folders/1GMArpxG8pPtqHECjsMXyaV_PoPayOiZz?usp=sharing